

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-acb020dace0672ede.jsonl`  
- SHA-256: `1ecf87b17c378fc1f8d6e6d041b2308dcd3ea2ab4034f6874b5fe50f8314f47a`  
- Source modified: `2026-06-16T03:25:40+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_3d536535-241`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T03:24:35.147Z)

Adversarial Claim Verifier (voter 3/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Research question: ****What concrete, recent (2022?2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 ? ~16 golden videos)?**** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):

- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96?99% per-frame visibility).

- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.

- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval?serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best ***recent*** methods for (a) turning a dense per-frame signal into well-segmented

actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMN/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k, edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RL00CV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible.

Claim under review

"On THUMOS14, TemporalMaxer achieves 67.7% average mAP (mAP@0.5=71.8, @0.7=44.7), beating the ActionFormer baseline (66.8% avg) while using 16.4 GMACs and only 7.1M parameters versus ActionFormer's 45.3 GMACs / 29.3M params ? i.e. a parameter-free temporal block roughly 4x smaller than the attention model still wins on accuracy."

****Source:**** <https://arxiv.org/abs/2303.09055> (primary)

****Supporting quote:**** "ActionFormer[60] I3D[7] 82.1 77.8 71.0 59.4 43.9 66.8 80 ...

Our(TemporalMaxer) I3D[7] 82.8 78.9 71.8 60.5 44.7 67.7 50"

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

****refuted=true**** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

****refuted=false**** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-16T03:24:38.182Z)

I'll verify this claim about TemporalMaxer's performance on THUMOS14. Let me check the primary

source.

3. assistant (2026-06-16T03:24:47.444Z)

I'll fetch the primary arXiv source to verify the specific numbers claimed.

4. user (2026-06-16T03:24:52.251Z)

I cannot extract the THUMOS14 results table you're requesting because it is not present in the provided web page content. The page shows only the arXiv abstract and metadata for the TemporalMaxer paper (arXiv:2303.09055), which states:

"Temporal Action Localization (TAL) is a challenging task in video understanding that aims to identify and localize actions within a video sequence."

The abstract mentions that TemporalMaxer "outperforms other state-of-the-art methods," but does not include specific numerical results, mAP values, GMACs, or parameter counts. To obtain the exact THUMOS14 results table with the specific metrics you need, you would need to access the full PDF of the paper via the provided link.

5. assistant (2026-06-16T03:24:54.592Z)

The abstract page doesn't contain the table. Let me fetch the full PDF and also search for corroborating sources.

6. user (2026-06-16T03:25:01.856Z)

I apologize, but I cannot extract the THUMOS14 results table you're requesting from the provided content. The document appears to be a PDF with heavily compressed/encoded stream data that doesn't render as readable text in the format provided.

The content shows PDF object streams with binary-encoded image and font data, but no accessible table with the specific TemporalMaxer and ActionFormer metrics you're looking for (mAP values, mAP@0.3-0.7, GMACs, and parameter counts).

To obtain these results, you would need to:

1. Access the original paper in a readable format (HTML or plaintext PDF)
2. Locate the THUMOS14 evaluation section
3. Find the results comparison table directly

If you can provide the paper in a more accessible text format, I'd be happy to help extract those specific numerical results.

[Binary content (application/pdf, 1.1MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\tool-results\webfetch-1781579965172-5ov5x2.pdf]

7. user (2026-06-16T03:25:03.335Z)

Web search results for query: "TemporalMaxer THUMOS14 67.7 average mAP ActionFormer 66.8 GMACs parameters"

Links: [{"title": "TemporalMaxer: Maximize Temporal Context with only Max Pooling", "url": "https://arxiv.org/pdf/2303.09055"}, {"title": "ActionFormer: Localizing Moments of Actions with Transformers", "url": "https://arxiv.org/pdf/2202.07925"}, {"title": ". . Latest updates: htps://dl.acm.org/doi/10.1145/3652628.3652695 .", "url": "https://dl.acm.org/doi/pdf/10.1145/3652628.3652695"}, {"title": "Benchmarking Data Efficiency and Computational Efficiency of Temporal Action Localization Models", "url": "https://arxiv.org/pdf/2308.13082"}, {"title": "GitHub - csh415/actionformer_performance", "url": "https://github.com/csh415/actionformer_performance"}, {"title": "ActionFormer: Localizing Moments of Actions with Transformers", "url": "https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136640485.pdf"}, {"title": "actionformer_release/README.md at main · minzhang15/actionformer_release", "url": "https://github.com/minzhang15/actionformer_release/blob/main/README.md"}, {"title": "Benchmarking Data Efficiency and Computational"}]

```
Efficiency","url":"https://cveu.github.io/2023/papers/11.pdf"},"title":"GitHub -  
happyharrycn/actionformer_release: Code release for ActionFormer (ECCV 2022) ·  
GitHub","url":"https://github.com/happyharrycn/actionformer_release"},"title":"(PDF)  
ActionFormer: Localizing Moments of Actions with Transformers","url":"https://www.researchgate.n  
et/publication/358655346_ActionFormer_Localizing_Moments_of_Actions_with_Transformers"]}]
```

Based on the search results, I found relevant information about the models and metrics you mentioned:

****TemporalMaxer on THUMOS14:****

TemporalMaxer with a kernel size of 3 achieved an average mAP of 67.7 on THUMOS14, which represents an improvement over ActionFormer's performance.

****ActionFormer on THUMOS14:****

ActionFormer reached state-of-the-art performance on the THUMOS'14 dataset obtaining an average mAP of 66.8%.

****Regarding GMACs and Parameters:****

While the search results mention that ActionFormer results reported include model complexity in GMACs, the specific GMACs and parameter values for both models were not detailed in the search results returned. The search results do indicate that TemporalMaxer was constructed to require a low computational cost without sacrificing localization performance by using a basic, parameter-free max pooling block on top of a pre-trained 3D CNN, suggesting it is more computationally efficient than ActionFormer.

For detailed GMACs and parameter specifications, you may need to consult the full papers directly, particularly the TemporalMaxer paper on arXiv (2303.09055).

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-16T03:25:06.673Z)

The search corroborates the headline mAP numbers (TemporalMaxer 67.7 avg, ActionFormer 66.8 avg). I have the PDF saved locally. Let me extract the table from it to verify the per-threshold values and especially the GMACs/parameter claims, which the web search couldn't confirm.

9. user (2026-06-16T03:25:15.090Z)

Requirement already satisfied: pycparser in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from cffi>=2.0.0->cryptography>=36.0.0->pdfminer.six==20251230->pdfplumber) (3.0)
=== PAGE 4 ===

Learning Objective. The model predicts ?? = 4.1.ResultsonTHUMOS14

t

(os,oe,c)foreverymomentoftheinputX. Followingthe
t t t Dataset. THUMOS14dataset[22]contains200valida-
baseline [60], the Focal Loss [31] and DIOU loss [64] are
tion videos and 213 testing videos with 20 action classes.
employedtosuperviseclassificationandregressionoutputs

Following previous work [29, 30, 54, 62, 60], we trained
respectively. Theoveralllossfunctionisdefinedas:

themodelusingvalidationvideosandmeasuredtheperfor-
L = (cid:88) (L +1 L)/T (7) manceontestingvideos.

total cls ct reg +

FeatureExtraction. Following[60,62],weextractthe

t

features of THUMOS14 dataset using two-stream I3D [7]

where L denotes regression loss and is applied only

reg pre-trainedonKinetics[24]. 16consecutiveframesarefed

when the indicator function, 1 , indicates that the current

ct intoI3Dpre-trainednetworkwithaslidingwindowofstride

timesteptisapositivesample.T isthenumberofpositive

+ 4. Theextractedfeatureiscollectedafterthelastfullycon-

samples.L isCwayclassificationloss.Thelossfunction

connected layer and has 1024-D feature space. After that, the L is applied to all levels on the output of multi-scale total two-stream features are further concatenated (2048-D) and feature pyramid Z and averaged across all video samples utilized as the input of the model.

during training.

Results. We compare the performance evaluated on the THUMOS14 dataset [22] with state-of-the-art meth-

4. Experimental Results

ods. TemporalMaxer demonstrates remarkable perfor-

In this section, we show that our proposed method, TemporalMaxer, achieves an average mAP of 67.7% mAP, out-

TemporalMaxer, demonstrates the outstanding results achieved performing all previous approaches, both single-stage, and

across a variety of challenging datasets, namely THUMOS two-stage methods, by a significant margin, with a 1.1% in-

[22], EPIC-Kitchens 100 [11], MultiTHUMOS [57], and

crease in mAP at $tIoU=0.4$. Especially, TemporalMaxer sur-

MUSES [34]. These datasets are recognized as standard passes all recent methods that utilize long-term TCM blocks

benchmarks in the Temporal Action Localization task. Our including self-attention such as TadTR [35], HTNet [23],

approach surpasses the state-of-the-art baseline, Action-

TAGS [38], GLFormer [19], ActionFormer [60], or Graph-

Former [60], in each dataset, showcasing its superior performance based like G-TAD [54], VSGN [61], or complex module

performance compared to other works. including Local-Global Temporal Encoder [40].

Evaluation Metric. We employ a widely-used evaluation metric for TAL known as the mean average precision (mAP) calculated at various temporal intersections exhibits outstanding performance but

also is efficient in over union (tIoU). tIoU is the intersection over union between two temporal windows, i.e., the 1D Jaccard index. takes 50ms on average to fully process

an entire video on

We report the mAP scores for all action categories based

THUMOS. It is 1.6x faster than the ActionFormer baseline

on the given tIoU thresholds, and further report an averaged and 3.9x faster than TadTR. However, in

section 4.5, we

mAP value across all tIoU thresholds. show that our model only takes 10.4ms for forward time. It

Training Details. To ensure a fair and unbiased comparison, we employed the experimental setup of the ro-

time-consuming. In the later section 4.5, we show that the

baseline model, ActionFormer [60]. This setup in-

forward time and the backbone time of four models are 2.9x

cluded various components such as decoder design d, and 8.0x faster than ActionFormer, respectively.

non-maximum suppression (NMS) hyper-parameters in the

4.2. Results on EPIC-Kitchens 100

post-processing stage, data augmentation, learning rate, fea-

ture extraction, and the number of feature pyramid levels. Dataset. The EPIC-

Kitchens 100 dataset [11] is a com-

The sole variation in our study was the substitution of the

prehensive collection of egocentric action videos, featuring

Transformer block in ActionFormer with the Max Pooling

100 hours of footage from 700 sessions that document cook-

block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thor-

Kitchens 100 is three times larger in

terms of total video

ough analysis of the effects of varying kernel sizes. During

hours and more than ten times larger in terms of action in-

training, the input feature length is kept constant at 2304,

stances(averaging128pervideo)whencomparedtoTHU-
correspondingtoapproximately5minutesofvideoonboth MOS14. Thesevideosarerecordedfromafirst-
personper-
THUMOS14andMultiTHUMOSdatasets,roughly20min-
spective,resultinginsignificantcameramotion,andrepre-
utesontheEPIC-Kitchens100dataset, andapproximately sentanovelchallengeforTALresearch.
45minutesonMUSES.Additionally,ModelEMA[21]and Feature Extraction. Following previous work [60,
29,
gradientclippingtechniquesareemployed,consistentwith 54], we extract the videos feature using
SlowFast network
thoseusedin[60],topromotetrainingstability. [15] pre-trained on EPICKitchens [11]. We utilized a
32-

=== PAGE 5 ===

tIoU?

Type Model Feature time(ms)?

0.3 0.4 0.5 0.6 0.7 Avg.

BMN[29] TSN[52] 56.0 47.4 38.8 29.7 20.5 38.5 483*

DBG[27] TSN[52] 57.8 49.4 39.8 30.2 21.7 39.8 ?

G-TAD[54] TSN[52] 54.5 47.6 40.3 30.8 23.4 39.3 4440*

BC-GNN[4] TSN[52] 57.1 49.1 40.4 31.2 23.1 40.2 ?

TAL-MR[62] I3D[7] 53.9 50.7 45.4 38.0 28.5 43.3 >644*

P-GCN[59] I3D[7] 63.6 57.8 49.1 ? ? ? 7298*

P-GCN[59]+TSP[2] R(2+1)1D[50] 69.1 63.3 53.5 40.4 26.0 50.5 ?

Two-Stage

TSA-Net[17] P3D[41] 61.2 55.9 46.9 36.1 25.2 45.1 ?

MUSES[34] I3D[7] 68.9 64.0 56.9 46.3 31.0 53.4 2101*

TCANet[40] TSN[52] 60.6 53.2 44.6 36.8 26.7 44.3 ?

BMN-CSA[45] TSN[52] 64.4 58.0 49.2 38.2 27.8 47.7 ?

ContextLoc[65] I3D[7] 68.3 63.8 54.3 41.8 26.2 50.9 ?

VSGN[61] TSN[52] 66.7 60.4 52.4 41.0 30.4 50.2 ?

RTD-Net[46] I3D[7] 68.3 62.3 51.9 38.8 23.7 49.0 >211*

Disentangle[66] I3D[7] 72.1 65.9 57.0 44.2 28.5 53.5 ?

SAC[55] I3D[7] 69.3 64.8 57.6 47.0 31.5 54.0 ?

A²Net[56] I3D[7] 58.6 54.1 45.5 32.5 17.2 41.6 1554*

GTAN[36] P3D[41] 57.8 47.2 38.8 ? ? ? ?

PBRNet[32] I3D[7] 58.5 54.6 51.3 41.8 29.5 ? ?

Single-Stage AFSD[28] I3D[7] 67.3 62.4 55.5 43.7 31.1 52.0 3245*

TAGS[38] I3D[7] 68.6 63.8 57.0 46.3 31.8 52.8 ?

HTNet[23] I3D[7] 71.2 67.2 61.5 51.0 39.3 58.0 ?

TadTR[35] I3D[7] 74.8 69.1 60.1 46.6 32.8 56.7 195*

GLFormer[19] I3D[7] 75.9 72.6 67.2 57.2 41.8 62.9 ?

AMNet[35] I3D[7] 76.7 73.1 66.8 57.2 42.7 63.3 ?

ActionFormer[60] I3D[7] 82.1 77.8 71.0 59.4 43.9 66.8 80

ActionFormer[60]+GAP[37] I3D[7] 82.3 ? 71.4 ? 44.2 66.9 >80

Our(TemporalMaxer) I3D[7] 82.8 78.9 71.8 60.5 44.7 67.7 50

Table1.TheresultsobtainedontheTHUMOS14dataset[22]arepresentedforvariousIoUthresholds,withtheaverageAPcalculated

intherange[0.3:0.7:0.1]. Thetop-performingresultsarehighlightedinbold.

Thetime(ms)istheaverageinferencetimeforonevideo,

without extracting features from 3D CNN and including the post-processing step, such as NMS. We measure the inference time using

asingleGeForceGTX1080TiGPU.Resultsindicatedwith*aretakenfrom[35]whicharemeasuredusingTeslaP100GPU,amuch

morepowerfulGPUthanthe1080Ti. Incomparisontoearlyworks,includingbothone-stageandtwo-stage methods, andthoseutilizing

long-termTCM, TemporalMaxerachie vessuperiorperformanceinbothmAPandinferencespeed.

frameinputsequencewithastrideof16togenerateasetof video hours with 25 action classes, designed to facilitate

2304-D features. These features were then fed as input to multi-shotanalysesmakingthedatasetchallenging.

ourmodel.

Result. Tab. 2 shows our results. TemporalMaxer

Feature Extraction. We directly employ the pre-demonstrates notable performance on the EPIC-Kitchens extractedfeatureprovidedby[34]. Thefeatureisextracted

100dataset, achieving an average mAP of 24.5% and 22.8% using a pre-trained I3D network [7] on the Kinetics dataset for verb and noun, respectively. The superiority of our approach [24] using only RGB stream, resulting in 1024-D feature space is further confirmed by a large margin over a strong baseline for video clip embedding.

and robust baseline, ActionFormer [60], with an average improvement of 1.0% mAP for verb and 0.9% mAP for noun. Again, TemporalMaxer outperforms other methods ResultTab. 3 shows the results on MUSES dataset. Our method that utilizes long-term TCM including self-attention [60] or Graph [54]. These results provide empirical evidence of long-term TCM such as G-TAD [54], Ag-Trans [63], and the effectiveness of the simplest backbone in advancing the ActionFormer [60] at every tIoU threshold. Notably, TemporalMaxer improves 1.0 mAP at tIoU=0.7. On average, we achieve 27.2 mAP, which is 1.0 mAP higher than the

4.3. Results on MUSES

previous approaches, demonstrating the robustness of our Dataset. The MUSES dataset [34] is a collection of method. It is worth noting that we implemented the ActionFormer [60] on the MUSES dataset using the code provided. MUSES has 31,477 action instances over a duration of 716 by the authors.

=== PAGE 7 ===

tIoU

TCM GMACs? #params(M)? time(ms)? backbone time(ms)?

0.5 0.7 Avg.

Conv[28](Our Impl) 62.8 37.1 59.4 45.6 30.5 16.3 9.0

Subsampling 64.3 37.7 61.0 16.2 7.1 10.4 2.5

Average Pooling 66.1 39.4 63.2 16.4 7.1 10.4 2.5

Transformer[51] 71.0 43.9 66.8 45.3 29.3 30.5 20.1

TemporalMaxer 71.8 44.7 67.7 16.4 7.1 10.4 2.5

Table 5. Ablation studies about different TCM blocks on THUMOS14.

Inference times are measured using an input video with 2304 clip embeddings, a 5-minute video, on a GeForce GTX 1080 Ti GPU without post-processing (NMS) and pre-extracted features step.

from 3D-CNN for the TAL task. PoolFormer retains the global information of adjacent clips is reduced. That is why

structure of Transformer such as FFN [43, 44], residual Average Pooling decreases by 3.6% mAP compared with

connection [18], and Layer Normalization [3] because the Transformer.

tokens have to be encoded by the PoolFormer itself. However, the ablation study with Average Pooling suggests that

ever, in TAL the features from 3D CNN have already been the most important information of clip embeddings should pre-extracted and contain useful information. Therefore, be maintained.

For that reason, we employ Max Pooling as

we posit that there will be a straightforward block that are

TCM. The result is provided in the last row of Tab. 5. Max

more efficient than Transformer/PoolFormer, does not re-

Pooling achieve the highest results at every tIoU threshold

quire much computational as well as parameters, and be and significantly outperforms the strong and robust base-

able to effectively exploit the pre-extracted features. line, ActionFormer. To clarify,

TemporalMaxer effectively

highlights only critical information from nearby clips and

Our ablation study starts to employ a 1-D convolution

discards the less important information. This result suggests

module as an alternative to Transformer [51] for TCM block

that the feature from pretrained 3D CNN are informative

to see how many mAP drop. The result is reported in the

and can be effectively utilized for the TAL model without

first row of Tab. 5. As expected, the convolution layer

complex modules like prior works. decreases by 7.4% mAP compared with the ActionFormer. Our method, TemporalMaxer, results in the simplest baseline. This deduction can be explained for two reasons. First, given the video clip features that are informative but more parameters and computational cost for the TAL model. TemporalMaxer is highly similar, and the convolution weight is fixed after training, consequently the convolution operation cannot outperform the robust baseline, ActionFormer, with 2.8x fewer GMACs and 3x faster inference speed. Especially, Second, the convolution layer introduces the most parameters when comparing only the backbone time, our proposed method only takes 2.5 ms which is incredibly 8.0x faster than ActionFormer backbone [63], 20.1 ms. trained 3D CNN, the model should not contain too many different values of kernel size. We make an ablation study with the kernel size of TemporalMaxer 3, 4, 5, 6 and the results are shown in Table 5. To further concretize our thought, we replace the average pooling with a parameter-free operation, subsampling. The results obtained for kernel sizes 3, 4, and 5 are not very different. This finding proves that TCM block may not need to be sensitive to the kernel size. This suggests that these kernel sizes can effectively capture the relevant temporal information. However, subsampling is prone to losing the most crucial information as only half of the video clip embeddings are kept after a TCM block. Thus, we replace the transformer with Average Pooling with kernel size 3 and stride 2. As expected, Average Pooling improves the result by 2.2% in mAP on average compared with the subsampling. It is because this operation does not drop any clip embedding simply maximize the underlying information in the video which helps retain the crucial information. However, the clip features from pre-trained 3D-CNN. To this end, with Average Pooling, we average the nearby features, thus the basic, non-parametric and temporally local operating max-

10. assistant (2026-06-16T03:25:28.400Z)

I have the full primary source data. Let me verify each component of the claim against the extracted tables.

****Table 1 (THUMOS14 main results):****

- ActionFormer [60] I3D: 82.1, 77.8, 71.0, 59.4, 43.9, ****66.8**** avg

- Our(TemporalMaxer) I3D: 82.8, 78.9, **71.8** (@0.5), 60.5, **44.7** (@0.7), **67.7** avg ?

****Table 5 (TCM ablation, GMACs/params):****

- Transformer[51]: @0.5=71.0, @0.7=43.9, Avg=66.8, GMACs=****45.3****, #params=****29.3M**** ?
- TemporalMaxer: @0.5=71.8, @0.7=44.7, Avg=67.7, GMACs=****16.4****, #params=****7.1M**** ?

Now let me check each part of the claim precisely:

1. "67.7% average mAP" ? ? Table 1
2. "mAP@0.5=71.8, @0.7=44.7" ? ? Table 1
3. "beating ActionFormer baseline (66.8% avg)" ? ? Table 1
4. "16.4 GMACs and only 7.1M parameters" ? ? Table 5 (TemporalMaxer row)
5. "versus ActionFormer's 45.3 GMACs / 29.3M params" ? ? Table 5 (Transformer row = ActionFormer's TCM)
6. "parameter-free temporal block roughly 4x smaller" ? $29.3M / 7.1M = 4.13x$?; $45.3/16.4 = 2.76x$ GMACs (paper text independently states "2.8x fewer GMACs")
7. "still wins on accuracy" ? ? $67.7 > 66.8$

One nuance worth noting: the GMACs/params in Table 5 come from the *ablation* (TCM block comparison), where "Transformer" = ActionFormer's design. The paper itself explicitly equates this row's performance (66.8 avg) with the ActionFormer baseline, and the body text states TemporalMaxer has "2.8x fewer GMACs" than ActionFormer, consistent with $45.3/16.4=2.76$. So attributing the 45.3 GMACs / 29.3M params to "ActionFormer" is the paper's own framing, not a misread.

Every specific number in the claim is verbatim-supported by the primary source. The "~4x smaller" characterization (params) and "wins on accuracy" are accurate. The web search independently corroborated the 67.7 vs 66.8 headline numbers.

The claim is fully supported, current (CVPRW 2023, an active and still-relevant TAL method), from the primary source, and not cherry-picked or marketing ? the numbers are the paper's own reported benchmark results, accurately transcribed.

11. user (2026-06-16T03:25:36.214Z)

Structured output provided successfully

12. assistant (2026-06-16T03:25:40.040Z)

Verification complete. The claim is fully supported by the primary source (arXiv:2303.09055, TemporalMaxer, CVPRW 2023). I extracted the actual PDF tables and confirmed every number verbatim: Table 1 gives TemporalMaxer 67.7 avg mAP (@0.5=71.8, @0.7=44.7) vs ActionFormer 66.8; Table 5 gives the GMACs/params (TemporalMaxer 16.4/7.1M vs ActionFormer-Transformer 45.3/29.3M). The "~4x smaller" ($29.3/7.1=4.13x$ params) and "wins on accuracy" characterizations are accurate, and WebSearch independently corroborated the headline figures. refuted=false, high confidence.